Intelligent Embedded Systems Lab

universität freiburg

Hahn Schickard

Embedded Systems Entrepreneurship

Evam Gilhotra (5584974), Hanzhi Zhuang (5578347)
Leon Mehl (5546576), Rushang Phira (5578541)

Health Dashboard

Date: August 6, 2023
Supervisors: Prof. Dr. Oliver Amft, Dijana Ivezic

Abstract

This project focuses on developing a health dashboard application with a focus on platform independence. The application is built using Flutter and Dart, and can be run on iOS and Android devices. It is compatible with any wearable device running wearOS. High-frequency health data is fetched from the device and its history is presented to the user. The main goal of this application is to provide detailed information about the user's physical condition, with a potential future in medical research.

Contents

1 Introduction

Smart watches have become increasingly popular in recent years, offering users a range of features and functionalities right on their wrists. Behind the sleek and compact design of these devices lies a sophisticated technology called embedded systems. Embedded systems play a crucial role in powering the functionality of smart watches, enabling them to perform a variety of tasks beyond just displaying the time.

1.1 Key Components of Embedded Systems in Smart Watches

1.1.1 Micro Controller Unit (MCU) or System on a Chip (SoC)

The heart of an embedded system in a smart watch is a micro controller unit or a system on a chip. It contains a processor, memory, and input/output peripherals. The MCU or SoC is responsible for executing the software instructions, controlling the hardware, and managing the overall system operation.

1.1.2 Sensors

Smart watches incorporate various sensors to collect data about the user and the environment. Common sensors found in smart watches include accelerometers, heart rate monitors, GPS, gyroscopes, ambient light sensors, and more. These sensors provide valuable input for fitness tracking, activity monitoring, sleep analysis, and other functionalities.

1.1.3 Display

Embedded systems in smart watches drive the display, which is usually a small, low-power touchscreen. The display enables users to interact with the watch, view notifications, access apps, and navigate through menus. Efficient management of power and screen brightness is essential for maximizing battery life.

1.1.4 Wireless Connectivity

Smart watches rely on wireless connectivity to synchronize data with smartphones, receive notifications, and access the internet. Embedded systems in smart watches support technologies like Bluetooth, Wi-Fi, and NFC, allowing seamless communication with other devices and networks.

1.1.5 Power Management

Given the limited size and battery capacity of smart watches, efficient power management is crucial. Embedded systems optimize power consumption by regulating the operation of various components, controlling backlighting, and implementing power-saving techniques, such as sleep modes and dynamic voltage scaling.

1.1.6 Operating System (OS)

Smart watches utilize an operating system to manage the software and provide a user interface. Popular operating systems for smart watches include Google's Wear OS, Apple's watchOS, and Samsung's Tizen. The embedded system interacts with the OS, executing applications, handling notifications, and ensuring smooth performance.

1.2 Applications of Embedded Systems in Smart Watches

1.2.1 Fitness and Health Tracking

Embedded systems enable smart watches to monitor heart rate, count steps, track sleep patterns, measure calories burned, and provide real-time fitness feedback. These features are particularly useful for individuals who want to maintain an active and healthy lifestyle.

1.2.2 Notifications and Communication

Embedded systems in smart watches receive and display notifications from smartphones, including calls, messages, emails, and social media alerts. Users can often respond to messages or initiate calls directly from their watches, thanks to the embedded systems' communication capabilities.

1.2.3 Personal Assistance and Voice Control

Many smart watches incorporate voice recognition and personal assistant features, such as Apple's Siri or Google Assistant. Embedded systems process voice commands, perform voice-to-text conversion, and execute various tasks based on user instructions.

1.2.4 Navigation and GPS

With built-in GPS functionality, smart watches equipped with embedded systems can provide navigation assistance, track outdoor activities like running or cycling, and display maps directly on the watch face.

2 Related work

2.1 Mobile Health Device

Recent studies show that the proportion of the population using smartphones is increasing. Approximately 72% of all adults in United States of America possess a smartphone. About 6.1 billion are estimated to be using a smartphone globally, covering around 80 percent of world's population [1]. With the spread of smartphone usage, changed peoples' lives have drastically changed, enabling them to have easier access to medical and fitness information, for example [2].

2.2 Common types of health apps

Currently over 165,000 mobile health applications (mHealth apps) are available for purchase or free of charge on major application stores, provided by developers from both professional medical centers and private companies [1]. Among these health applications, fitness and self-monitoring ones are the most common, taking up a proportion of 74.8% [2]. Health resources and calory intake are the next two most popular categories. These mobile health applications are used in unsupervised and stand-alone mode by most users. Self-goal tracking, eliminating unhealthy habits and acquiring awareness of health and fitness are the tasks often done by them [3].

2.3 Potentials and Problems

The potential for mHealth apps to offer up-to-date, affordable and high-quality health information is enormous. With its comprehensive record of health data, these applications can have a great impact on health-related domains like chronic disease, mental health, patient education, etc. The major motivation and target of developing a mHealth application is to enhance quality, diminish cost and improve accessibility for providing accurate health data and insightful health information. However, lack of formal vetting processes and well-rounded clinical evidence in these applications prevent the health care providers from recommending the mHealth apps to their patient for medical treatment. The user experience is also not highly rated [1].

2.4 Current Solutions

According to Sama et al., the majority of contemporary mHealth applications focus on health monitoring and body fitness. At this time, there are few methods to get users more engaged, which means there is still room for improvement. The paper suggests that investments in the scientific and development communities as well as financial motivations to develop a particular advanced

and all-rounded application might change the market's dynamics and have an influence on patient health outcomes if the health data can be utilized more efficiently and feedback could be more targeted. This move is currently done by major Health Service Providers (HSPs), who are building their own health service for smartphone and wearable device to deliver supervised health service [2].

2.4.1 Apple health

Apple Health is a health and fitness application that is developed by Apple. It displays an overview of all health statistics in one app. Apple Health is aimed at people who want to have a better understanding of their health and fitness condition. It allows the user to gain a complete overview of healthcare statistics in one application [4].

The Health app collects your health and fitness data from compatible applications and devices to create a visual dashboard in its current state. With a browse tab, source tab and medical ID tab, Apple Health presents all relevant health facts and information in a nice and user-friendly interface.

In terms of connectivity and compatibility, Apple Health has excellent compatibility with Apple devices. Other than that, only a few certified third-party wearable devices are able to work with Apple health [3]. Connection to it is very system and device-dependent.

2.4.2 Android health applications

Google is another giant health service provider. Google's health service is called Google Fit. All health data is stored in Google Fit Cloud and can be utilized through the Google Fit Website portal or through a Representational State Transfer API. It provides an approach for medical service providers, such as hospitals, to access the fitness data [3]. Another way to access the fitness data is through an application developed by Google called Google Fit App, targeting Android device users.

2.5 Current Problems for developing health application for wearable devices

Almost all Android smartphone manufacturers equip their products with their self-developed health applications for measuring and displaying health data.

A majority of these manufacturers and third-party manufacturers also make wearable devices. The problem with these devices is that a majority of them connect to a phone with its own health application developed by the manufacturer. There does not exist a general channel or a universal solution for

connecting wearable devices with different phones. For this reasons, manufacturers create their own applications, which are only suitable for their own wearable devices. As a result, most wearable devices are software-dependent, which means they work well only with their proprietary application. This leads to users needing to download a new application for connecting for every wearable device from different brands. The downside for wearable device developers is that they had to maintain separate versions of the same application for different operating systems.

2.6 Cross-platform Development Environment

Luckily, software developing framework suitable for multi-platforms have emerged. Because an increasing amount of developers are attracted by the benefits brought by it, many new cross-platform mobile development technology have gained popularity.

Nowadays, cross-platform development frameworks are widely used by mobile engineers to build native-looking applications for multiple platforms with a single code base. Having a shareable code base is one of the key advantages over native app development. This allows developers to save time and cost, accelerating the development process eventually [5].

According to Statistics, "Flutter", "React Native" and "Cordova" are the three most popular cross-platform development frameworks favored by developers from 2019 to 2022. Among them, Flutter is the most popular cross-platform mobile framework used by global developers, 46% of software developers are using Flutter [6].

The Flutter framework, simple yet powerful, has been used to develop many applications. From sports applications such as "EntrenaPro" and "Top Goals", to the shopping application "Alibaba", as well as Business applications like "Google Ads", Flutter frameworks are widely chosen by developers around the world[7].

3 Methods

3.1 Hardware and frameworks

3.1.1 Hardware

The Suunto 7 smartwatch serves as the primary device for collecting data, and is capable of monitoring heart rate, count steps, analyze sleep schedules, detect calories and workout activity. Bluetooth and Wi-Fi connectivity enables the user to synchronize the watch with smartphones.

Another feature of the Suunto 7 is the operating system. WearOS provides access to a wide range of apps and services that are available on the Google Play Store. The integration of WearOS on the watch enables easy access to health data, notifications and messages on any mobile device that supports Google service.

3.1.2 Google Fit API

Because the watch is loaded with Wear OS and supports Google service, Google Fit API can be used on the device. Google Fit can integrate data from multiple resources and it offers a free and effective solution to track and improve health data.

The Google Fit APIs integrated within Google Play services pertain to Android. These APIs are compatible with Android 4.1 (API level 16) and later versions. By utilizing these APIs, the following features can be achieved in the application:

- Fetch both current and past data, including information from Bluetooth Low Energy (BLE) devices, in nearly real-time
- Record various types of activities
- Link data with a specific session
- Define fitness goals

3.1.3 Flutter and Android Studio

For the development environment of this project, Flutter and Android Studio are used, along with Dart. Flutter is a platform-independent framework which can generate code to run on any system (Android, iOS, Web) with little to no change of existing code. It equips with its own render engine that can utilize almost all functions on both Android and iOS. Android Studio is a powerful IDE (Integrated Development Environment) for Android applications. With a Gradle-Build system, device emulator debug support, multiple programming language support and build-in git support, this IDE offers an array of powerful

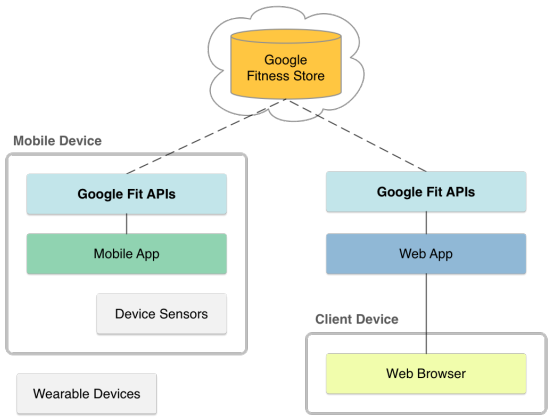

Figure 1: Google Fit on Mobile Device
[8]

tools and features for us to build, test, and optimize the health dashboard application.

3.1.4 Data Fetching

Data is sent from the watch to Google cloud and synchronized every 30 minutes. Our application receives data from this cloud database.

3.2 Code Implementation

In order to read and write health data from and to Apple Health and Google Fit, "Health 6.0.0" plugin is used in our project. This plugin has prepared wrapped health data types and methods to use actual health data from device. It is the main channel used in our project to communicate between Flutter and Google Fit.

In order to access data from Google Fit on our own laptops for development, we created a Google account specifically for this project. We then added the SHA1 key of our laptops to the Google account, so that an OAuth 2.0 client ID is obtained. This permits our devices (laptop) to run the application. Google Play Service Fitness library is then added to the dependencies, after which we declared the necessary permissions required to access health data in `AndroidManifest.xml`. The software environment is now set up for development.

3.3 Structure of the code

3.3.1 "main.dart" File

The Flutter application can be run by setting the Dart entrypoint to `./lib/main.dart`, in which, lists of data points (Heart rate, energy burned, steps, sleep analysis, workout time and distance) and single data points are initialized as static variables. When the widget initializes its state during running, three functions will be executed sequentially. First, a function called "authorize()" is run to await for the permission request to access the data. After that, a function called "fetchall()" will call other functions which fetch each kind of health data from Google Fit and store them in the static variables declared before.

- fetchall()
  The `fetchall()` function consists of six functions, that are "fetchStepData()", "fetchHeartRate()", "fetchactiveenergyburned()", "fetchBloodPressure()", "fetchSleepData()" and "fetchDistanceDelta()". These six functions are used to fetch the specific kind of data to the Flutter code.
- fetchStepData()
  In the "fetchStepData()" function, the start and the end dates of the last seven days are first calculated, after authorization is made, the step data in the interval of last seven days is requested from Google Fit via Health package, and then it is stored in a list structure. The total foot steps of each day is then calculated and stored in variables. Because connection to Google fit to fetch data is the main process of the function, which may take longer time than the user interface is loaded, these data needs be waited. Therefore, the function is an asynchronous function and returns a data type "Future". And the functions that calls the asynchronous functions should also be declared "async" and returns a "Future" data type, as is the case in "fetchall()" and "performBackgroundFetching()" function.

  For the rest five functions called by "fetchall()" function, which are "fetchHeartRate()", "fetchactiveenergyburned()", "fetchBloodPressure()", "fetchSleepData()" and "fetchDistanceDelta()", they share similar structure and way of implementation with the "fetchStepData()" function. After authorization, health data in the interval of the past seven days are fetched via health package and then stored in a list variable for future use.
- performBackgroundFetching()
  Finally, "performBackgroundFetching()" is performed. It is a method to execute "fetchall()" in every 15 minutes' interval. Therefore, the health data will technically be refreshed on the application every 15 minutes.

```
1 void performBackgroundFetching() async{
2   // Configure the background fetch
3     BackgroundFetch.configure(
```

```
4       BackgroundFetchConfig(
5         minimumFetchInterval: 15, // Fetch interval in minutes
6         stopOnTerminate: false,
7         enableHeadless: true,
8         startOnBoot: true,
9         requiresBatteryNotLow: false,
10        requiresCharging: false,
11        requiresStorageNotLow: false,
12      ), (String taskId) async {
13        await fetchAll();
14        BackgroundFetch.finish(taskId);
15      },
16    ).then((int status) {
17      print('[BackgroundFetch] configure success: $status');
18    }).catchError((e) {
19      print('[BackgroundFetch] configure ERROR: $e');
20    });
21  }
```

After initializing the widget states, an override of build function is performed to create the home page widget. The layouts of the entire widget is written in function ”fetchall()”, called in the ”body” element of build function.

The ”fetchall()” function basically determines the layout of the main page, in which, fonts, icons and containers are placed and colored in an organized way. In addition, containers of the six displayed health data is set to be able to route to the detailed page of the health data history.

3.4 Widget Files

The six dart files ”Steps.dart”, ”SleepScheduleWidget.dart”, ”CaloriesWidget.dart”, ”HeartRateWidget.dart”, ”TrainingDataWidget.dart”, ”WorkoutWidget.dart” are created to build the user interface for the detailed history page of the health data.

3.5 Data Acquisition

In order to create health data on the watch, one of our teammates wear the watch from time to time to get the health data, such as the sleep routine, walking steps and other data that can be recorded by the watch. In this way, data will be generated and sent to Google Fit. Data displayed on Google Fit are always used to assess whether the data we fetched to GUI is valid or not. Because chances are that the statistics displayed on our application are different from the real ones, if the methods we created to retrieve data are not working in the right way.

4 Results

4.1 Layouts

When first connecting to a physical device or emulator to run the project, the application will be built, which takes a long time. During the process, the application is installed on that device. After the program is successfully built, the home page of our application will be initiated.

4.1.1 Home page

On the top of our home page is the name of application, which is "Health Dashboard". Below that is a big container. On the top of the big container, there is a sentence that welcomes the user to the application. After the welcome sentence, there lie six small round-cornered square widgets, for each of which, six different types of health data read from Google Fit is displayed, as well as the icons of the health data. Tapping the widgets will lead to their respective detailed pages.

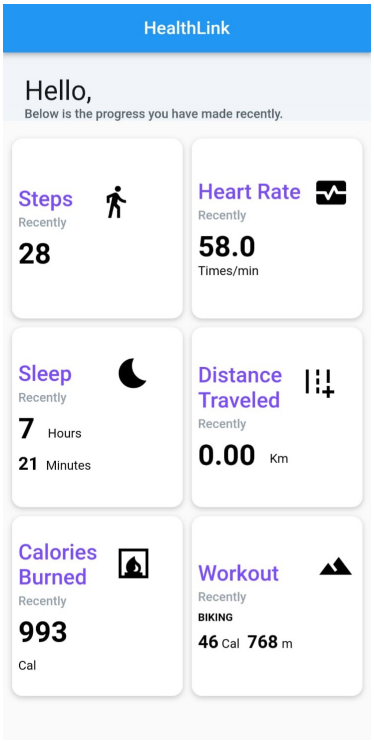

Figure 2: Home Page

4.1.2 Steps

On the top of this page, there is a line chart showing the step data of the past seven days. Detailed statistics of the step data (total steps of the day, Maximum step of the week, Minimum of the week and average step of the week) are displayed below the graph.

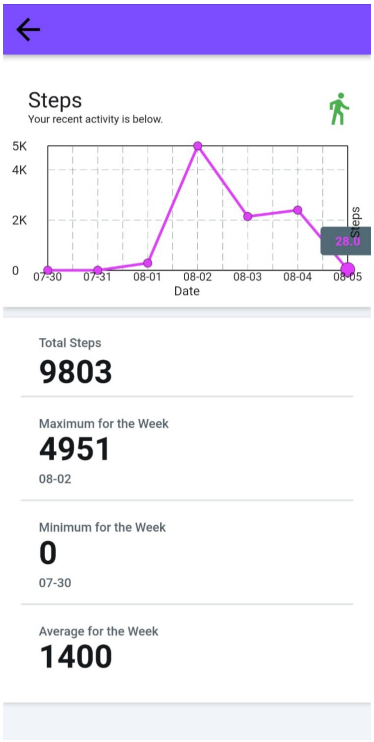

Figure 3: Steps

4.1.3 Heart Rate

On the top of this page, there are three line charts showing the minimum, maximum and average heart rate data of the past seven days. Detailed statistics of heart rate data are displayed below the graph.

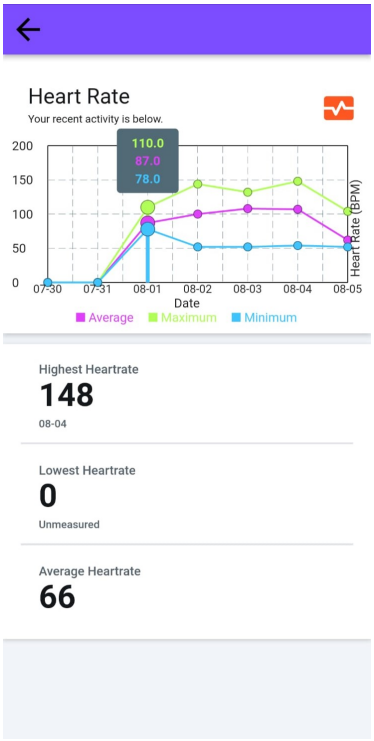

Figure 4: Heart Rate

4.1.4 Sleep

On the top of this page, there is a line chart showing the sleep duration of the past seven days. Detailed statistics of the sleep data, such as average sleeping time, longest sleeping time and shortest sleeping time, are displayed below the graph.

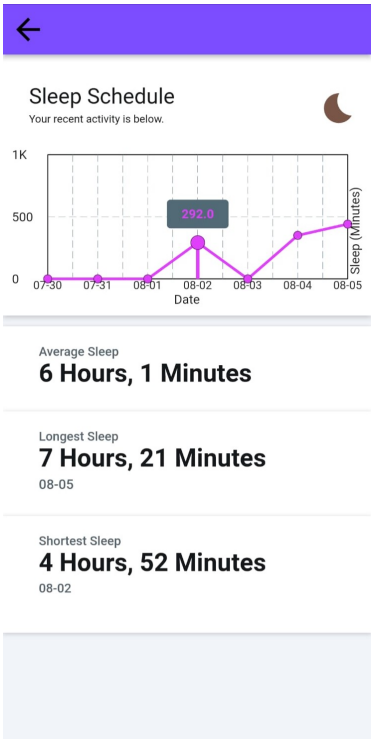

Figure 5: Sleep Schedule

4.1.5 Distance Traveled

On the top of this page, there is a line chart showing the distance traveled of the past seven days. Total distance, maximum daily travel distance of the week, minimum of the week and average travel distance of the week are displayed below the graph.

Figure 6: Distance Traveled

4.1.6 Calories Burned

On the top of this page, there is a line chart showing the calories burned of the past seven days. Detailed statistics of the calories data, such as most calories burned, lowest calories burned and average calories burned are displayed below the graph.

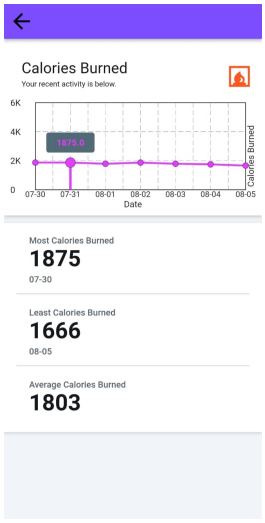

Figure 7: Calories Burned

4.1.7 Workout

On the top of this page, there is two line charts showing the calories burned and distance traveled data for the workout of the past seven days. Detailed statistics of the workout data, such as total distance covered this week and total calories burned this week are displayed below the graph.

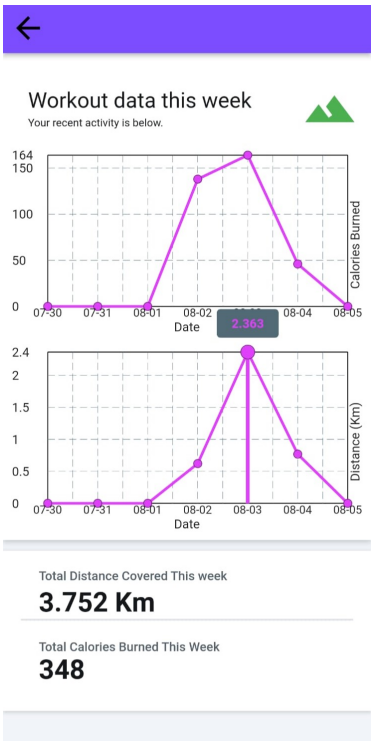

Figure 8: Workout

5 Discussion

There are some limitations present in the application. The `performBackgroundFetching()` function sets the `minimumFetchInterval` attribute to 15. This allows it to refresh every 15 minutes. However, it is bottlenecked by the 30-minute window that Google Fit takes to receive the most recent data from the smartwatch. Unfortunately, the frequency cannot be increased on the current version of WearOS.

```
22 try {
23         authorized =
24         await health.requestAuthorization(types, permissions: permissions);
25       } catch (error) {
26         print("Exception in authorize: $error");
27       }
```

6 Conclusion and further work

6.1 Conclusions

In this work, we developed a cross-platform application based on a platform independent framework (Flutter) and utilized information channels between both a smartwatch and a smartphone for data acquisition via Flutter package and Google Fit. User can use this application to keep track of their health data through time, understanding in which areas can they improve and how they did improve over time, in terms of steps, heart rate, sleep schedule, workout time, calories burned and distance traveled. The highlight of the application is the usage of the platform independent framework, which enable the system to run on any system with little to no change of existing code, relative high frequency of data acquisition rate which in turn provides valuable information for potential use in medical research.

6.2 Future works

The user interface and performance of the application will improved in the future, so that users can have a better interaction experience, more types of health data recorded and more frequent acquisition of data from data source.

In the future, Data could be processed to provide users with recommendation systems and explanations of their current state as a part of a future research project, the dashboard could provide users with interesting details of what is going on with their health. Medical personnel can interpret data presented in the dashboard, estimate overall state of their patients and provide recommendations and nudges.

The application will possibly be improved and hopefully launched in major app stores in the future. Chances are that cooperation with wearable companies will be made to develop tailored versions of application which fit in their products.

Appendix

.1 Configurations

The application we built is working successfully under the environment:

- Flutter: 3.7.12
- Dart: 2.19.6
- Java Development kit (JDK): 13.0.2
- Android Studio: 2022.2

With dependencies:

- cupertino_icons:1.0.2
- permission_handler: 10.2.0
- health: 6.0.0
- flutter_background_service: 3.0.1
- background_fetch: 1.1.6
- fl_chart: 0.50.6
- google_fonts: 2.2.0
- percent_indicator: 4.2.2.

With test devices:

- Google Pixel 3a (Emulator)
- Samsung Android smart phone

Data Source Device:

- Suunto Watch 7

Code Listings

References

[1] C.-K. Kao and D. M. Liebovitz, "Consumer mobile health apps: Current state, barriers, and future directions," vol. 9, no. 5, pp. S106–S115. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1934148217303829

[2] P. R. Sama, Z. J. Eapen, K. P. Weinfurt, B. R. Shah, and K. A. Schulman, "An evaluation of mobile health application tools," vol. 2, no. 2, p. e3088, company: JMIR mHealth and uHealth Distributor: JMIR

mHealth and uHealth Institution: JMIR mHealth and uHealth Label: JMIR mHealth and uHealth Publisher: JMIR Publications Inc., Toronto, Canada. [Online]. Available: https://mhealth.jmir.org/2014/2/e19

[3] B. A. Farshchian and T. Vilarinho, "Which mobile health toolkit should a service provider choose? a comparative evaluation of apple HealthKit, google fit, and samsung digital health platform," in *Ambient Intelligence*, ser. Lecture Notes in Computer Science, A. Braun, R. Wichert, and A. Maña, Eds. Springer International Publishing, pp. 152–158.

[4] Apple health app and HealthKit: What are they and how do they work? Section: Apps. [Online]. Available: https://www.pocket-lint.com/apps/news/apple/131130-apple-healthkit-and-health-app-how-they-work-and-the-medical-records-you-need/

[5] Cross-platform mobile development: Five best frameworks | SaM solutions. Section: Mobile Development. [Online]. Available: https://www.sam-solutions.com:443/blog/cross-platform-mobile-development/

[6] The six most popular cross-platform app development frameworks | kotlin. [Online]. Available: https://kotlinlang.org/docs/cross-platform-frameworks.html

[7] https://www.facebook.com/droidsonroids. Top apps made with flutter – 17 stories by developers and business owners. Section: Blog. [Online]. Available: https://www.thedroidsonroids.com/blog/apps-made-with-flutter

[8] Platform overview | google fit | google for developers. [Online]. Available: https://developers.google.com/fit/overview

List of Figures

List of Tables